INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	08/11/2026
Enlace de Git	https://github.com/diyertorres-commits/repositorio_base-datos.git
Nombres y apellidos del estudiantes	Diyer Arley Torres Troya

1. TITULO

Diseño e implementación de una base de datos relacional para la gestión de una tienda de aplicaciones y juegos

2. CONTEXTO DEL PROBLEMA

Las tiendas de aplicaciones digitales son plataformas donde las personas descubren, compran y descargan software, ya sea en forma de juegos o de herramientas de uso cotidiano (educación, productividad, utilidades). En este entorno intervienen varios actores: los desarrolladores que publican sus aplicaciones, los usuarios que las adquieren y las valoran, y la propia tienda, que debe organizar el catálogo por categorías y llevar el control de las versiones que se van lanzando.

Sin una base de datos, esta información tendría que manejarse en hojas de cálculo o archivos sueltos. Eso provoca datos duplicados o inconsistentes (por ejemplo, un mismo desarrollador escrito de varias formas), dificulta saber qué versión de una aplicación está vigente, qué compras realizó cada usuario o cuántas veces se descargó una aplicación, y vuelve lento y propenso a errores cualquier consulta o reporte.

El problema afecta a los usuarios, que necesitan encontrar aplicaciones confiables y consultar su historial de compras y descargas; a los desarrolladores, que requieren saber cómo se reciben sus aplicaciones a través de las descargas y las reseñas; y a la administración de la tienda, que necesita información confiable para controlar las ventas y el catálogo.

Es necesario gestionar los datos de las aplicaciones (nombre, descripción, precio, tamaño, clasificación por edad y fecha de publicación), diferenciando los juegos de las herramientas, junto con sus desarrolladores, categorías y versiones; y también los datos de los usuarios, sus compras (con el detalle de las aplicaciones adquiridas y el precio pagado), las descargas realizadas y las reseñas con su calificación.

3. OBJETIVOS

3.1. Objetivo General

Diseñar e implementar una base de datos relacional para una tienda de aplicaciones y juegos, a partir del modelo Entidad-Relación y del lenguaje SQL, para organizar de forma consistente la información de aplicaciones, desarrolladores, usuarios, compras, descargas y reseñas.

3.2. Objetivos Específicos

1. Elaborar el diagrama Entidad-Relación de la tienda (con una generalización para juegos y herramientas, y una entidad débil para las versiones) y derivar de él el modelo relacional con sus claves primarias y foráneas.
2. Construir el esquema en SQL (DDL), poblarlo con datos de ejemplo (DML) y formular cinco consultas en álgebra relacional traducidas a SQL, para comprobar que el diseño responde a las necesidades de información del sistema.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

Modelo Entidad-Relación: tienda de aplicaciones y juegos

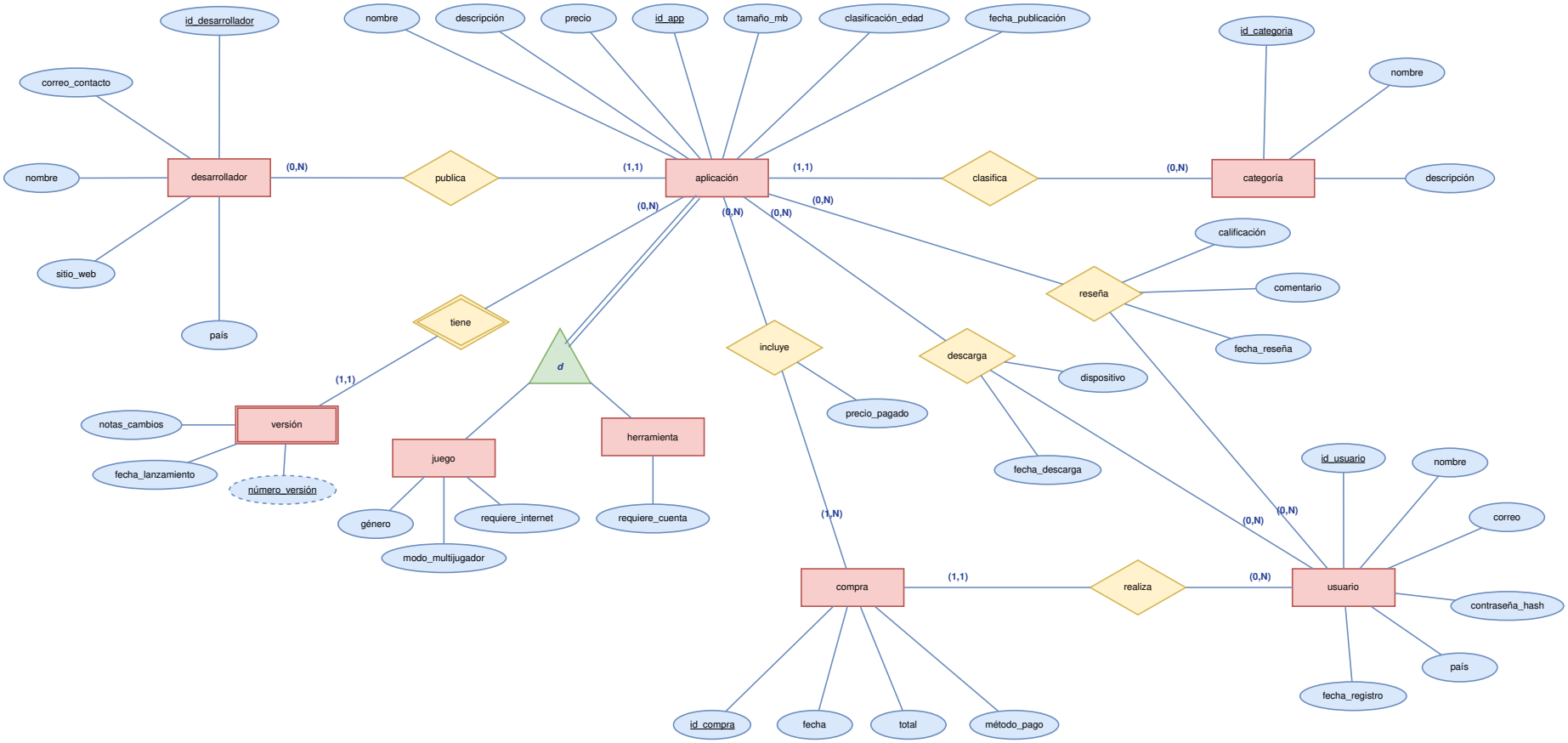


Figura 1. Diagrama Entidad-Relación del sistema de tienda de aplicaciones y juegos.

4.2. Modelo Relacional

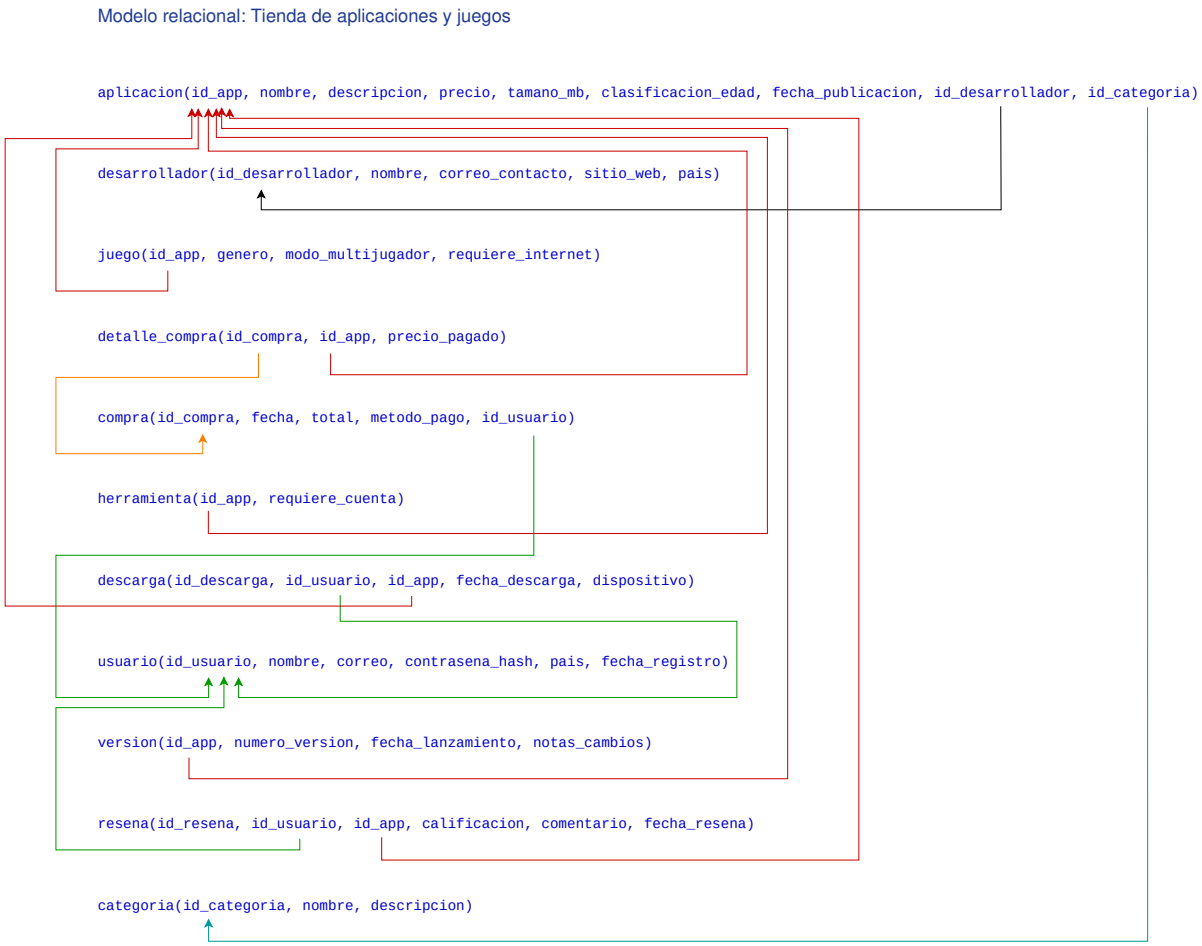


Figura 2. Modelo relacional de la tienda de aplicaciones y juegos (PK: clave primaria, FK: clave foránea; las flechas van de la FK a la tabla referenciada).

CÓDIGO SQL

5.1. Creación de Tablas (DDL)

A continuación se presenta el script completo (MySQL 8) que crea las 11 tablas del modelo, en un orden que respeta las dependencias entre llaves foráneas.

```
-- Base de datos de la tienda de aplicaciones y juegos
-- Proyecto: Tienda de aplicaciones y juegos

CREATE TABLE desarrollador (
    id_desarrollador VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo_contacto VARCHAR(100) NOT NULL,
    sitio_web VARCHAR(150),
    pais VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE categoria (
    id_categoria VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(255)
);

CREATE TABLE usuario (
    id_usuario VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL UNIQUE,
    contrasena_hash VARCHAR(255) NOT NULL,
    pais VARCHAR(50) NOT NULL,
    fecha_registro DATE NOT NULL
);

CREATE TABLE aplicacion (
    id_app VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion VARCHAR(500),
    precio DECIMAL(8,2) NOT NULL,
    tamano_mb DECIMAL(10,2) NOT NULL,
    clasificacion_edad VARCHAR(10) NOT NULL,
    fecha_publicacion DATE NOT NULL,
    id_desarrollador VARCHAR(15) NOT NULL,
    id_categoria VARCHAR(15) NOT NULL,
    FOREIGN KEY (id_desarrollador) REFERENCES desarrollador(id_desarrollador),
    FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)
);

CREATE TABLE juego (
    id_app VARCHAR(15) PRIMARY KEY,
    genero VARCHAR(50) NOT NULL,
    modo_multijugador BOOLEAN NOT NULL,
    requiere_internet BOOLEAN NOT NULL,
    FOREIGN KEY (id_app) REFERENCES aplicacion(id_app)
);

CREATE TABLE herramienta (
    id_app VARCHAR(15) PRIMARY KEY,
    requiere_cuenta BOOLEAN NOT NULL,
    FOREIGN KEY (id_app) REFERENCES aplicacion(id_app)
);

CREATE TABLE version (
    id_app VARCHAR(15),
    numero_version VARCHAR(20),
    fecha_lanzamiento DATE NOT NULL,
    notas_cambios VARCHAR(500),
    PRIMARY KEY (id_app, numero_version),
    FOREIGN KEY (id_app) REFERENCES aplicacion(id_app)
);

CREATE TABLE compra (
    id_compra VARCHAR(15) PRIMARY KEY,
    fecha DATE NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    metodo_pago VARCHAR(30) NOT NULL,
    id_usuario VARCHAR(15) NOT NULL,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

CREATE TABLE detalle_compra (
    id_compra VARCHAR(15),
```

```
    id_app VARCHAR(15),
    precio_pagado DECIMAL(8,2) NOT NULL,
    PRIMARY KEY (id_compra, id_app),
    FOREIGN KEY (id_compra) REFERENCES compra(id_compra),
    FOREIGN KEY (id_app) REFERENCES aplicacion(id_app)
);

CREATE TABLE descarga (
    id_descarga VARCHAR(15) PRIMARY KEY,
    id_usuario VARCHAR(15) NOT NULL,
    id_app VARCHAR(15) NOT NULL,
    fecha_descarga DATE NOT NULL,
    dispositivo VARCHAR(50),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_app) REFERENCES aplicacion(id_app)
);

CREATE TABLE resena (
    id_resena VARCHAR(15) PRIMARY KEY,
    id_usuario VARCHAR(15) NOT NULL,
    id_app VARCHAR(15) NOT NULL,
    calificacion INT NOT NULL CHECK (calificacion BETWEEN 1 AND 5),
    comentario VARCHAR(500),
    fecha_resena DATE NOT NULL,
    UNIQUE (id_usuario, id_app),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_app) REFERENCES aplicacion(id_app)
);
```

5.2. Inserción de Datos (DML)

Se insertan datos coherentes entre sí (cada clave foránea corresponde a un registro existente) en las 11 tablas, con al menos cinco registros en las tablas principales, para que las consultas posteriores devuelvan resultados significativos.

```
-- Datos de ejemplo (se insertan en orden de dependencia)
-- 5.2 Insercion de Datos (DML): Tienda de aplicaciones y juegos
-- Se insertan en orden de dependencia (primero las tablas sin FK)

INSERT INTO desarrollador (id_desarrollador, nombre, correo_contacto, sitio_web, pais)
VALUES
    ('DEV01', 'Andes Studio', 'contacto@andesstudio.ec', 'www.andesstudio.ec', 'Ecuador'),
    ('DEV02', 'Loja Soft', 'info@lojasoft.ec', 'www.lojasoft.ec', 'Ecuador'),
    ('DEV03', 'Pixel Norte', 'hola@pixelnorte.co', 'www.pixelnorte.co', 'Colombia'),
    ('DEV04', 'Cóndor Labs', 'soporte@condorlabs.pe', 'www.condorlabs.pe', 'Perú'),
    ('DEV05', 'Iberia Apps', 'contacto@iberiaapps.es', 'www.iberiaapps.es', 'España');

INSERT INTO categoria (id_categoria, nombre, descripcion)
VALUES
    ('CAT01', 'Acción', 'Juegos de acción y reflejos'),
    ('CAT02', 'Aventura', 'Juegos narrativos y de exploración'),
    ('CAT03', 'Educación', 'Aplicaciones para aprender y estudiar'),
    ('CAT04', 'Productividad', 'Herramientas de organización y trabajo'),
    ('CAT05', 'Utilidades', 'Herramientas de uso diario');

INSERT INTO usuario (id_usuario, nombre, correo, contrasena_hash, pais, fecha_registro)
VALUES
    ('USR01', 'Ana Mora', 'ana.mora@correo.com', 'hash_a1f3c9', 'Ecuador', '2025-01-20'),
    ('USR02', 'Luis Pérez', 'luis.perez@correo.com', 'hash_b72e10', 'Ecuador', '2025-03-14'),
    ('USR03', 'María Gómez', 'maria.gomez@correo.com', 'hash_c58d44', 'Colombia', '2025-05-
02'),
    ('USR04', 'Carlos Ruiz', 'carlos.ruiz@correo.com', 'hash_d90a17', 'Perú', '2025-08-19'),
    ('USR05', 'Sofía Vega', 'sofia.vega@correo.com', 'hash_e31b86', 'España', '2026-01-08'),
```

```
    ('USR06', 'Diego Salinas', 'diego.salinas@correo.com', 'hash_f46c52', 'Ecuador', '2026-
03-25');

INSERT INTO aplicacion
    (id_app, nombre, descripcion, precio, tamano_mb, clasificacion_edad,
     fecha_publicacion, id_desarrollador, id_categoria)
VALUES
    ('APP01', 'Aventura Andina', 'Acertijos en los Andes', 4.99, 320.50, '+7', '2025-03-10',
'DEV01', 'CAT02'),
    ('APP02', 'Carrera Veloz', 'Carreras arcade', 0.00, 210.00, 'Todos', '2025-06-18',
'DEV03', 'CAT01'),
    ('APP03', 'Guerreros del Sol', 'Combate por turnos', 2.99, 480.75, '+12', '2025-09-02',
'DEV04', 'CAT01'),
    ('APP04', 'Misterio en Loja', 'Novela de detectives', 3.49, 150.20, '+12', '2026-01-15',
'DEV02', 'CAT02'),
    ('APP05', 'Cóndor Runner', 'Corredor de montaña', 0.00, 95.30, 'Todos', '2026-04-22',
'DEV01', 'CAT01'),
    ('APP06', 'Tareas Pro', 'Gestor de tareas', 1.99, 25.40, 'Todos', '2025-02-05', 'DEV02',
'CAT04'),
    ('APP07', 'Estudia Fácil', 'Tarjetas de estudio', 0.00, 38.10, 'Todos', '2025-08-20',
'DEV05', 'CAT03'),
    ('APP08', 'Calculadora Científica', 'Funciones avanzadas', 0.99, 12.60, 'Todos', '2025-
11-11', 'DEV03', 'CAT05'),
    ('APP09', 'Notas Seguras', 'Notas con contraseña', 2.49, 18.90, 'Todos', '2026-02-28',
'DEV04', 'CAT04'),
    ('APP10', 'Idiomas Rápido', 'Inglés y quichua', 0.00, 64.00, 'Todos', '2026-05-30',
'DEV05', 'CAT03');

-- Especializacion de aplicacion: cada app va en juego O en herramienta
INSERT INTO juego (id_app, genero, modo_multijugador, requiere_internet)
VALUES
    ('APP01', 'Puzles', FALSE, FALSE),
    ('APP02', 'Carreras', TRUE, TRUE),
    ('APP03', 'Estrategia', TRUE, TRUE),
    ('APP04', 'Misterio', FALSE, FALSE),
    ('APP05', 'Corredor infinito', FALSE, FALSE);

INSERT INTO herramienta (id_app, requiere_cuenta)
VALUES
    ('APP06', FALSE),
    ('APP07', TRUE),
    ('APP08', FALSE),
    ('APP09', TRUE),
    ('APP10', TRUE);

-- Entidad debil: la PK es (id_app, numero_version)
INSERT INTO version (id_app, numero_version, fecha_lanzamiento, notas_cambios)
VALUES
    ('APP01', '1.0.0', '2025-03-10', 'Lanzamiento inicial'),
    ('APP01', '1.1.0', '2025-07-01', 'Nuevos niveles y corrección de errores'),
    ('APP02', '1.0.0', '2025-06-18', 'Lanzamiento inicial'),
    ('APP02', '1.2.0', '2025-12-05', 'Modo multijugador en línea'),
    ('APP03', '1.0.0', '2025-09-02', 'Lanzamiento inicial'),
    ('APP04', '1.0.0', '2026-01-15', 'Lanzamiento inicial'),
    ('APP06', '1.0.0', '2025-02-05', 'Lanzamiento inicial'),
    ('APP06', '2.0.0', '2025-10-14', 'Rediseño de la interfaz y recordatorios'),
    ('APP07', '1.0.0', '2025-08-20', 'Lanzamiento inicial'),
    ('APP09', '1.0.0', '2026-02-28', 'Lanzamiento inicial');

-- El total de cada compra es la suma de su detalle
INSERT INTO compra (id_compra, fecha, total, metodo_pago, id_usuario)
VALUES
    ('CMP01', '2025-04-02', 6.98, 'Tarjeta', 'USR01'),
    ('CMP02', '2025-09-20', 2.99, 'PayPal', 'USR02'),
    ('CMP03', '2026-02-02', 4.48, 'Tarjeta', 'USR03'),
    ('CMP04', '2026-03-10', 7.48, 'Transferencia', 'USR04'),
```

```
    ('CMP05', '2026-04-05', 3.98, 'Tarjeta', 'USR01'),
    ('CMP06', '2026-05-12', 5.98, 'PayPal', 'USR05');

INSERT INTO detalle_compra (id_compra, id_app, precio_pagado)
VALUES
    ('CMP01', 'APP01', 4.99),
    ('CMP01', 'APP06', 1.99),
    ('CMP02', 'APP03', 2.99),
    ('CMP03', 'APP04', 3.49),
    ('CMP03', 'APP08', 0.99),
    ('CMP04', 'APP01', 4.99),
    ('CMP04', 'APP09', 2.49),
    ('CMP05', 'APP03', 2.99),
    ('CMP05', 'APP08', 0.99),
    ('CMP06', 'APP04', 3.49),
    ('CMP06', 'APP09', 2.49);

INSERT INTO descarga (id_descarga, id_usuario, id_app, fecha_descarga, dispositivo)
VALUES
    ('DES01', 'USR01', 'APP01', '2025-04-02', 'Samsung Galaxy A54'),
    ('DES02', 'USR01', 'APP06', '2025-04-02', 'Samsung Galaxy A54'),
    ('DES03', 'USR02', 'APP03', '2025-09-20', 'Xiaomi Redmi Note 12'),
    ('DES04', 'USR02', 'APP02', '2025-10-01', 'Xiaomi Redmi Note 12'),
    ('DES05', 'USR03', 'APP04', '2026-02-02', 'Motorola Edge 30'),
    ('DES06', 'USR03', 'APP02', '2026-02-03', 'Motorola Edge 30'),
    ('DES07', 'USR04', 'APP07', '2025-08-25', 'Huawei P40'),
    ('DES08', 'USR04', 'APP01', '2026-03-10', 'Huawei P40'),
    ('DES09', 'USR05', 'APP04', '2026-05-12', 'iPhone 13'),
    ('DES10', 'USR05', 'APP07', '2026-06-02', 'iPhone 13'),
    ('DES11', 'USR05', 'APP10', '2026-06-02', 'iPhone 13'),
    ('DES12', 'USR06', 'APP05', '2026-04-25', 'Samsung Galaxy A14'),
    ('DES13', 'USR06', 'APP02', '2026-04-26', 'Samsung Galaxy A14');

-- Una sola reseña por usuario y app (UNIQUE)
INSERT INTO resena (id_resena, id_usuario, id_app, calificacion, comentario, fecha_resena)
VALUES
    ('RES01', 'USR01', 'APP01', 5, 'Muy buenos acertijos', '2025-05-10'),
    ('RES02', 'USR01', 'APP06', 4, 'Útil para organizar tareas', '2025-05-12'),
    ('RES03', 'USR02', 'APP03', 4, 'Buena estrategia, algo lenta', '2025-10-05'),
    ('RES04', 'USR02', 'APP02', 3, 'Divertido, pero con fallos', '2025-10-20'),
    ('RES05', 'USR03', 'APP04', 5, 'Historia muy atrapante', '2026-02-20'),
    ('RES06', 'USR04', 'APP01', 4, 'Falta más contenido', '2026-04-01'),
    ('RES07', 'USR05', 'APP04', 2, 'Se cierra en mi equipo', '2026-05-20'),
    ('RES08', 'USR06', 'APP05', 5, 'Adictivo y gratuito', '2026-05-02'),
    ('RES09', 'USR06', 'APP02', 4, 'Muy buenos gráficos', '2026-05-05');

-- Verificacion rapida de cantidades
SELECT 'desarrollador' AS tabla, COUNT(*) AS filas FROM desarrollador
UNION ALL SELECT 'categoria', COUNT(*) FROM categoria
UNION ALL SELECT 'usuario', COUNT(*) FROM usuario
UNION ALL SELECT 'aplicacion', COUNT(*) FROM aplicacion
UNION ALL SELECT 'juego', COUNT(*) FROM juego
UNION ALL SELECT 'herramienta', COUNT(*) FROM herramienta
UNION ALL SELECT 'version', COUNT(*) FROM version
UNION ALL SELECT 'compra', COUNT(*) FROM compra
UNION ALL SELECT 'detalle_compra', COUNT(*) FROM detalle_compra
UNION ALL SELECT 'descarga', COUNT(*) FROM descarga
UNION ALL SELECT 'resena', COUNT(*) FROM resena;
```

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

A continuación se presentan cinco consultas formuladas primero en álgebra relacional y luego traducidas a SQL. Tienen distinto nivel de complejidad (de una a tres tablas) y emplean operadores diferentes:

selección y proyección, reunión natural (JOIN), diferencia, división y agrupamiento con funciones de agregación. Los resultados esperados corresponden a los datos de ejemplo de la sección 5.2.

Consulta 1 — Selección y Proyección (operadores: σ, π)

Descripción: obtener el nombre y el tamaño de las aplicaciones gratuitas que pesan menos de 100 MB, por ejemplo para recomendarlas a usuarios con poco espacio en el dispositivo.

Álgebra Relacional:

```
π nombre, tamano_mb ( σ precio == 0 and tamano_mb < 100 (aplicacion))
```

SQL:

```
SELECT nombre, tamano_mb FROM aplicacion WHERE precio = 0 AND tamano_mb < 100;
```

Resultado esperado: tres aplicaciones: Cóndor Runner (95.30 MB), Estudia Fácil (38.10 MB) e Idiomas Rápido (64.00 MB). Carrera Veloz es gratuita, pero pesa 210 MB, por lo que no cumple la condición.

nombre	tamano_mb
Cóndor Runner	95.3
Estudia Fácil	38.1
Idiomas Rápido	64.0

Consulta 2 — Reunión natural / JOIN de tres tablas (operador:)

Descripción: listar los juegos con modo multijugador, indicando su género y el desarrollador que los publicó. Combina las tablas APLICACION, JUEGO y DESARROLLADOR a través de sus claves foráneas.

Álgebra Relacional:

```
π nombre, genero, nombre_dev ( σ modo_multijugador == 1 ( (aplicacion ▫ juego) ▫ ρ
nombre  nombre_dev (desarrollador) ) )
```

SQL:

```
SELECT a.nombre AS aplicacion, j.genero, d.nombre AS desarrollador
FROM   aplicacion a
JOIN   juego j         ON a.id_app = j.id_app
JOIN   desarrollador d ON a.id_desarrollador = d.id_desarrollador
WHERE  j.modo_multijugador = TRUE;
```

Resultado esperado: dos juegos: Carrera Veloz (Carreras, Pixel Norte) y Guerreros del Sol (Estrategia, Cóndor Labs).

nombre	genero	nombre_dev
Carrera Veloz	Carreras	Pixel Norte
Guerreros del Sol	Estrategia	Cóndor Labs

Consulta 3 — Diferencia (operador: −)

Descripción: encontrar los usuarios registrados que nunca han realizado una compra. Se restan los identificadores de usuarios con compras del conjunto de todos los usuarios y luego se recuperan sus datos.

Álgebra Relacional:

```
π nombre, correo ( usuario ▫ ( π id_usuario (usuario) - π id_usuario (compra) ) )
```

SQL:

```
SELECT nombre, correo
FROM   usuario
WHERE  id_usuario NOT IN (SELECT id_usuario FROM compra);
```

Resultado esperado: un usuario: Diego Salinas (diego.salinas@correo.com), que solo ha descargado aplicaciones gratuitas.

nombre	correo
Diego Salinas	diego.salinas@correo.com

Consulta 4 — División (operador: ÷)

Descripción: obtener los usuarios que han descargado todas las aplicaciones de la categoría Educación (id_categoria = 3). La división selecciona a los usuarios que están relacionados con todas las aplicaciones de ese conjunto.

Álgebra Relacional:

```
π nombre ( usuario ▫ ( π id_usuario, id_app (descarga) ÷ π id_app ( σ id_categoria == 'CAT03'
(aplicacion) ) ) )
```

SQL:

```
SELECT u.nombre
FROM   usuario u
WHERE  NOT EXISTS (
         SELECT a.id_app
         FROM   aplicacion a
         WHERE  a.id_categoria = 'CAT03'
           AND  NOT EXISTS (
                  SELECT 1
                  FROM   descarga d
                  WHERE  d.id_usuario = u.id_usuario
                    AND  d.id_app = a.id_app
                )
);
```

Resultado esperado: un usuario: Sofía Vega, la única que descargó tanto Estudia Fácil como Idiomas Rápido (las dos aplicaciones de Educación).

nombre
Sofía Vega

Consulta 5 — Intersección (operador: ∩)

Descripción: obtener las aplicaciones que tienen al menos una reseña con calificación de 4 o más y que además han sido compradas por algún usuario. La intersección conserva solo los id_app que aparecen en ambos conjuntos.

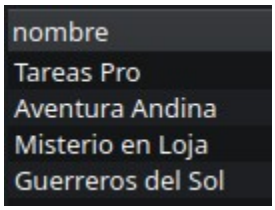
Álgebra Relacional:

```
π nombre ( aplicacion ▫ ( π id_app ( σ calificacion >= 4 ( resena ) ) ∩ π id_app
( detalle_compra ) ) )
```

SQL:

```
SELECT a.nombre
FROM   aplicacion a
WHERE  a.id_app IN (SELECT id_app FROM resena WHERE calificacion >= 4)
  AND  a.id_app IN (SELECT id_app FROM detalle_compra)
ORDER BY a.nombre;
```

Resultado esperado: cuatro aplicaciones: Aventura Andina, Guerreros del Sol, Misterio en Loja y Tareas Pro. Cóndor Runner y Carrera Veloz tienen reseñas de 4 o más, pero son gratuitas y nunca se compraron, así que no aparecen.



nombre
Tareas Pro
Aventura Andina
Misterio en Loja
Guerreros del Sol

CONCLUSIONES

- **Conclusión 1:** el modelo Entidad-Relación permitió traducir la problemática de la tienda en entidades, atributos y relaciones concretas, definiendo bien las cardinalidades (por ejemplo, la relación M:N entre compra y aplicación, con su atributo precio_pagado) y eliminando atributos redundantes. La generalización (Juego/Herramienta) y la entidad débil (Versión) representaron con precisión casos que con entidades simples quedarían ambiguos, y las reglas de transformación llevaron con claridad a las 11 tablas del modelo relacional.
- **Conclusión 2:** escribir el DDL y el DML mostró cómo se materializa el diseño en un gestor relacional. Las restricciones (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK) hacen que el propio sistema cumpla reglas como precios no negativos, calificaciones de 1 a 5 o correo único por usuario. También se comprobó que la disjunción total de la generalización no se puede declarar con restricciones simples y requiere lógica adicional.
- **Conclusión 3:** formular primero las consultas en álgebra relacional facilitó entender qué operación se necesita antes de escribir el SQL: selección y proyección en WHERE y SELECT, reunión en JOIN, diferencia en NOT IN, división en el doble NOT EXISTS e intersección en dos condiciones IN combinadas con AND. Las consultas más útiles fueron las que combinan varias tablas, porque entregan información para decidir que no está en una sola tabla.

BIBLIOGRAFÍA

Referencias bibliográficas

[1] M. L. Gillenson, *Fundamentals of Database Management Systems*, 3.ª ed. Wiley, 2024.

[2] J. A. Hoffer, R. Venkataraman, H. Topi y H. Bala, *Modern Database Management*, 14.ª ed. Pearson, 2025.

[3] C. Coronel y S. Morris, *Database Systems: Design, Implementation, & Management*, 14.ª ed. Cengage, 2023.

[4] J. L. Viescas, *SQL Queries for Mere Mortals: A Hands-On Guide to Data Manipulation in SQL*, 4.ª ed. Pearson.

[5] Oracle Corporation, *MySQL 8.4 Reference Manual*. [En línea]. Disponible en: https://dev.mysql.com/doc/refman/8.4/en/

[6] draw.io Ltd., «About draw.io». [En línea]. Disponible en: https://www.drawio.com/docs/about/

[7] P. P. Chen, «The entity-relationship model—toward a unified view of data», *ACM Transactions on Database Systems*, vol. 1, n.º 1, pp. 9–36, mar. 1976. https://doi.org/10.1145/320434.320440

[8] E. F. Codd, «A relational model of data for large shared data banks», *Communications of the ACM*, vol. 13, n.º 6, pp. 377–387, jun. 1970. https://doi.org/10.1145/362384.362685